UNIVERSIDAD NACIONAL DE SAN AGUSTIN

ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS

Laboratorio:

Funciones lambda

e introducción a Widget App Qt

I. OBJETIVOS

- Comunicaciones entre objetos widgets del SDK Qt.
- Conocer sobre funciones lambda en C++.

II. TEMAS A TRATAR

- Implementación de funciones lambda
- Comunicación entre objetos widgets

III. MARCO TEORICO

A.- Funciones Lambda:

- Las funciones lambda son expresiones que nos permiten escribir funciones anónimas en una sola línea, con lo que se evita la declaración de funciones adicionales.
- Los lambda puede combinarse con la librería STL de algoritmos.

La sintaxis completa es:
[capture_block](parameters) mutable exception_specification attribute_specifier -> return_type {body}

- **Capture Block:** especifica como las variables de enfoque son capturadas y puestas a disposición por la función lambda.
- **Parameters (opcional):** lista de parámetros para la función lambda.
- Mutable (opcional): especifica que la función no será constante (non-const).
- **Exception_specification (opcional)**: puede ser usada para especificar qué expresión del cuerpo de la función puede ser lanzada.
- **attribute specifier (opcional)**: puede ser usada para especificar atributos para la expresión lambda.
- **Return_type (opcional)**: es el tipo de valor a ser retornado.

Para declarar una función lambda:

- Puede ser a través de puntero a función.
  ```
  int (*s)(int) = [](int valor){ return valor;}
  ```
- Puede ser declarado por medio de la palabra reservada “auto”

```
auto R  = [](int valor){ return valor;}
```

parámetros:
[=] permite usar las variables externas pero sin ser modificadas en el lambda.
[&] permite usar las variables externas y si permite modificarlas dentro del lambda.
[this] captura el objeto circundante. En el cuerpo de la expresión lambda se puede acceder al objeto que apunta el puntero 'this', incluso sin usar 'this->'.

B.- Signals & Slots

- SEÑAL es un mensaje enviado por un objeto, es la respuesta a un evento.
- SLOT O RANURA: es una función que se llamará cuando se active una "señal".
- La función de CONEXIÓN especifica qué señal está vinculada con qué ranura/slot.

Nomenclatura de la conexión SIGNAL & SLOT:
connect(sender, &Sender::signalName, receiver, &Receiver::slotName)
connect(sender, &Sender::signalName, [function lambda])

IV. ACTIVIDADES

a) Funciones lambda

Función que sume un número y lo muestre

```
int (*s)(int) = [](int valor){ return valor+3;};
qDebug() << s(u) <<endl;
```

Función para recorrer elementos de un vector con el lambda

```
vector<int> v {5,2,3,7,1,0,14,20};
for_each(v.begin(),v.end(), [](int valor){
   qDebug() << valor << endl;
});
```

b) 2da parte: Aplicación Qt Widgets, signals & slots en objetos de formulario GUI

(IDE QT Creator)

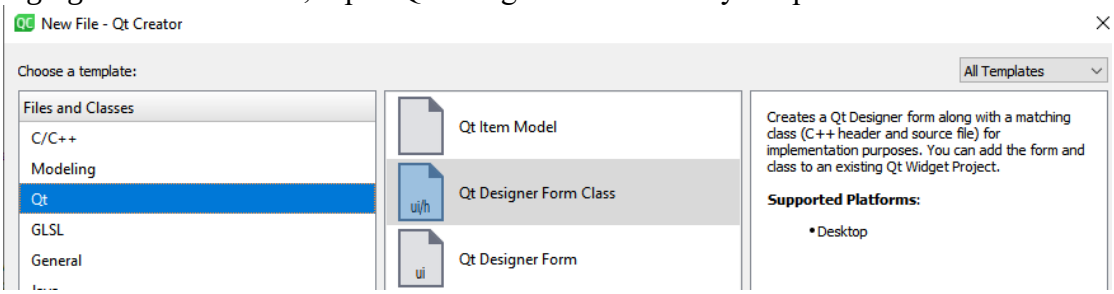
Editar el archivo MainWindow.ui

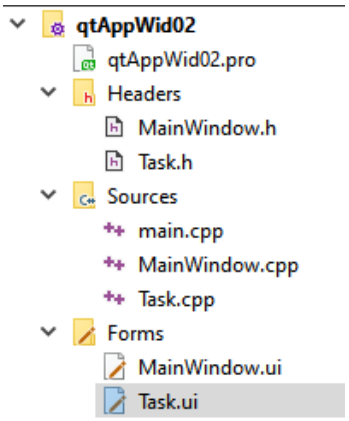
Agregar los siguientes objetos:

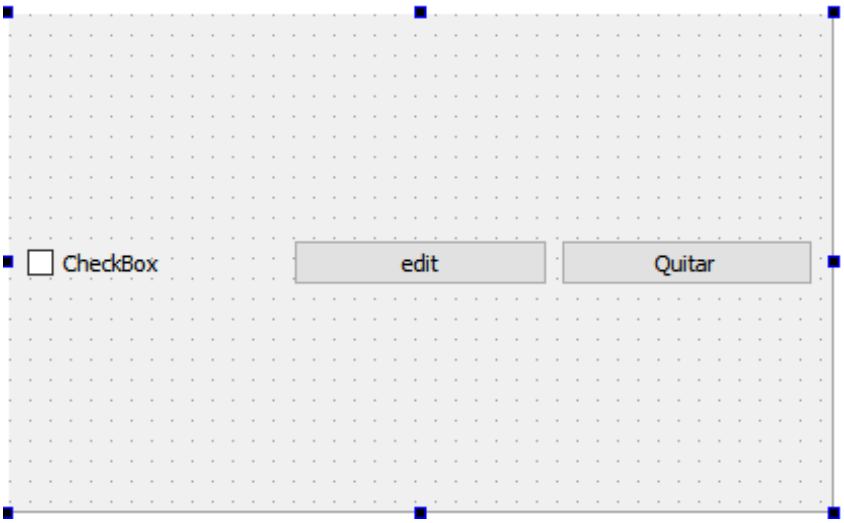
Label → statusLabel	propiedad text: Checklist de tareas
Button → addTaskButton	propiedad text: Agregar tarea
Vertical layout → tasksLayout	propiedad text:



Agregar la clase ‘Task’, tipo “Qt Dedigner form class” y template WIDGET.







En el formulario Task.ui agregar los objetos
checkbox (objectName = checkbox)
text (objectName = Mitext)

En el formulario QWidget, clic derecho y escoger ordenamiento vertical (layout → horizontally)

Notas:

- MainWindows.ui → contiene 3 objetos, un checkbox, un Botton (push buttom) y un Layout horizontal.
  Estos objetos debemos direccionar funcionalidad por signal y slot.
  El objeto Layout servirá de contenedor para guardar y mostrar los objetos de la clase Tarea.
- La clase Task hereda de la clase QWidget y mantiene Q_OBJECT para optimizar el uso de la memoria.
  Al ser un QWidget, las propiedades de la clase Task pueden transferirse hacia el objeto Layout de MainWindow.

- *ui es un puntero del tipo MainWindows, servirá para instanciar objetos del formulario ui.
- MainWindow.cpp implementa las Signals y Slots. En el constructor se debe implementar la conexión entre los objetos "ui" con las señales y slots.
  Para el caso, se requiere personalizar la llamada del clic (signal) del botón del formulario hacia un slot particular:
  - ✓ el slot debe escribir en la consola debugging.
  - ✓ el slot debe permitir instanciar objetos de la clase 'Task', e insertar el objeto en el layout.
  - ✓ Los objetos Task instanciados se guardan en una QVector cada vez que se hace clic en el botón.
  - ✓ Se debe implementar un slot que llame a un signal de la clase Task para remover (destruir) el objeto.
- En formulario 'Task.ui', debe definirse un signal para emitir la eliminación de un objeto Task, esto desde el clicked del botón 'quitar'.

1.- MainWindow.cpp, en el constructor agregar el vector como parámetro.

*MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent), ui(new Ui::MainWindow), mTasks()*

```
connect(ui->addTaskButton, &QPushButton::clicked, this, &MainWindow::addTask);
```

Compilar y ejecutar. Cuando damos clic en el botón la aplicación hace Quit (es el slot).

2.- Slots personalizados, en el MainWindow.h se debe agregar:

```
public slots:
    void addTask();

 void removeTask(Task* task);

private:
    Ui::MainWindow *ui;
    QVector<Task*> mTasks;
```

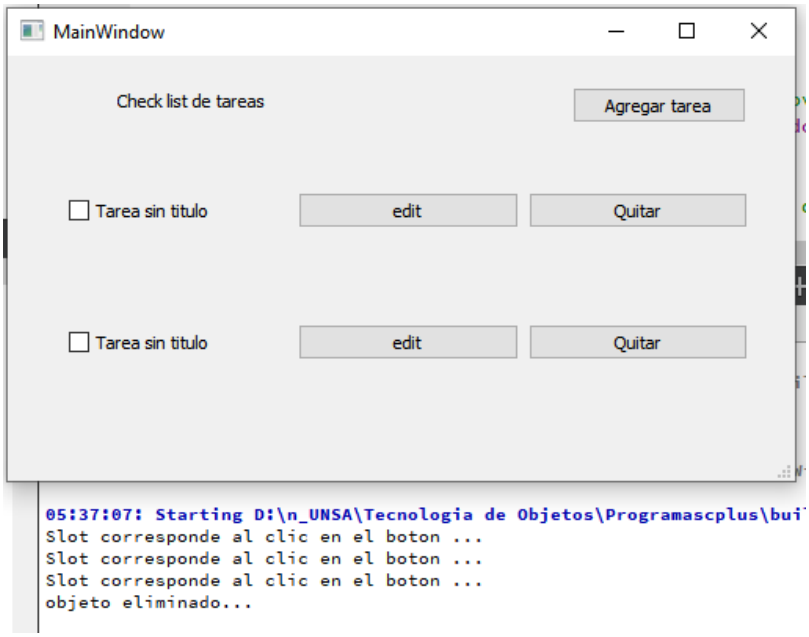
3.- archivo cabecera ‘Task.h’, definir el signal para recibir la llamada para eliminar un objeto instanciado.

```
signals:
    void removed(Task* task);
```

4.-Constructor del ‘Task.cpp’.

```
connect(ui->removeButton, &QPushButton::clicked, [this]{emit removed(this);});
```

5.- Ahora, ejecutar la aplicación y, hacer clic en el botón y debe escribir en la consola.



VII. Ejercicios

1. Contar los números impares y menores de 20 de un vector de números enteros generados de forma aleatoria (la solución puede usar el método std :: count_if)

2. Diferencias, ventajas y desventajas de las clases QVector y QList.

3. Crear una interface gráfica (que implemente señales y slots) que muestre una lista de países, al dar clic sobre alguno que se muestre un Label o Text con el idioma y capital correspondiente.

Referencias:

- https://www.qt.io/
- QT creator manual